

Lab Manual: Introduction to ADO.NET in Blazor Server

Getting Started with ADO.NET and Database Connectivity in Blazor Server

Lab Objectives:

- Understand the basics of ADO.NET and its components.
- Learn how to connect a Blazor Server app to SQL Server using ADO.NET.
- Perform basic CRUD operations (Create, Read, Update, Delete).
- Build a simple **Employee Management System**.

Introduction to ADO.NET

What is ADO.NET?

ADO.NET (ActiveX Data Objects for .NET) is a data access technology used to interact with databases in .NET applications. It provides a bridge between the front-end UI and the backend database.

Key Components:

Component	Description
SqlConnection	Establishes a connection to the database.
SqlCommand	Executes SQL queries and stored procedures.
SqlDataReader	Reads a stream of rows from a SQL Server database.
SqlDataAdapter	Used for filling DataSet and updating the database.
DataTable	In-memory representation of tabular data.

Prerequisites

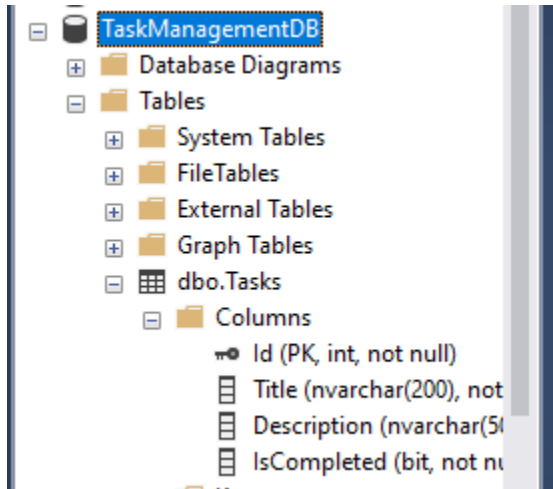
Before starting:

- Install **SQL Server Express** and **SSMS**.

- Install .NET SDK.
- Use **Visual Studio 2022 or newer** with Blazor Server template.

Walkthrough Task — Connecting to a Database

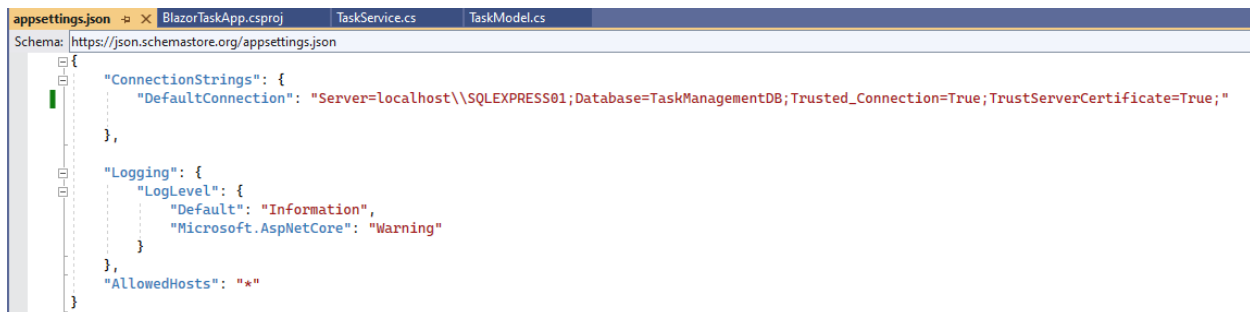
Step1: Create database and table



	Id	Title	Description	IsCompleted
1	6	Practice ADO.NET	CRUD via SSMS	1
2	7	Learn Blazor	Components & pages	0
3	8	Practice ADO.NET	CRUD via SSMS	0

Step 2: Add Connection String

Open appsettings.json and add:

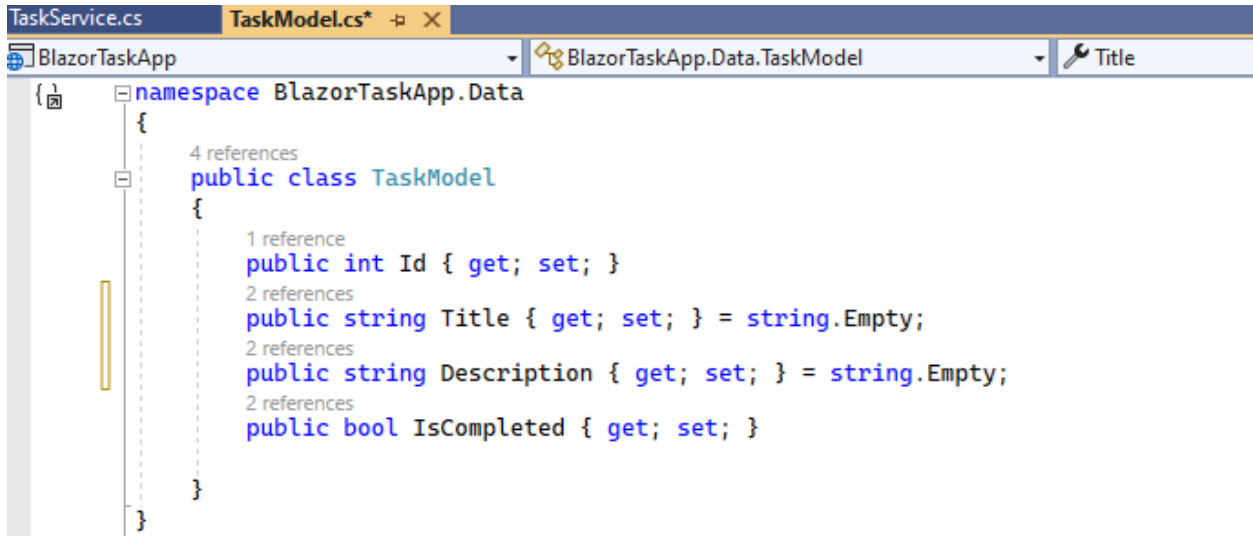


Replace localhost\\SQLEXPRESS if your server instance name is different.

Step3:

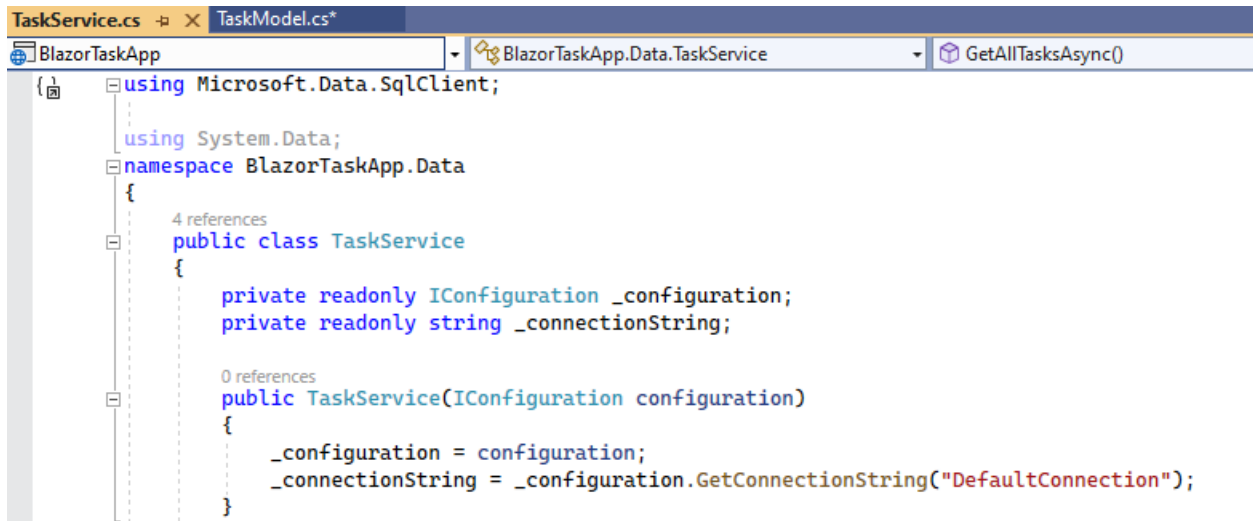
Create a Data Access Services

Service1: TaskModel.cs



```
TaskService.cs TaskModel.cs* X
BlazorTaskApp BlazorTaskApp.Data.TaskModel Title
{
    namespace BlazorTaskApp.Data
    {
        4 references
        public class TaskModel
        {
            1 reference
            public int Id { get; set; }
            2 references
            public string Title { get; set; } = string.Empty;
            2 references
            public string Description { get; set; } = string.Empty;
            2 references
            public bool IsCompleted { get; set; }
        }
    }
}
```

Service 2: TaskService.cs



```
TaskService.cs X TaskModel.cs*
BlazorTaskApp BlazorTaskApp.Data.TaskService GetAllTasksAsync()
{
    using Microsoft.Data.SqlClient;
    using System.Data;
    namespace BlazorTaskApp.Data
    {
        4 references
        public class TaskService
        {
            private readonly IConfiguration _configuration;
            private readonly string _connectionString;

            0 references
            public TaskService(IConfiguration configuration)
            {
                _configuration = configuration;
                _connectionString = _configuration.GetConnectionString("DefaultConnection");
            }
        }
    }
}
```

```

1 reference
public async Task<List<TaskModel>> GetAllTasksAsync()
{
    var tasks = new List<TaskModel>();
    using SqlConnection conn = new(_connectionString);
    await conn.OpenAsync();
    var cmd = new SqlCommand("SELECT * FROM Tasks", conn);
    using var reader = await cmd.ExecuteReaderAsync();
    while (await reader.ReadAsync())
    {
        tasks.Add(new TaskModel
        {
            Id = reader.GetInt32(0),
            Title = reader.GetString(1),
            Description = reader.IsDBNull(2) ? "" : reader.GetString(2),
            IsCompleted = reader.GetBoolean(3)
        });
    }
    return tasks;
}
}
}

```

Step4: Register service in program.cs

```
builder.Services.AddScoped<TaskService>();
```

Step5: create razor component and Inject service in Tasklist.razor page



```

Tasklist.razor*  Program.cs  TaskService.cs  TaskModel.cs*
@using BlazorTaskApp.Data
@page "/tasks"
@inject TaskService taskService

<h3>Tasks</h3>

@if (error != null)
{
    <p class="text-danger">@error</p>
}
else if (tasks == null)
{
    <p>Loading...</p>
}
else
{
    <ul>
        @foreach (var task in tasks)
        {
            <li>@task.Title - @task.Description (@(task.IsCompleted ? "Done" : "Pending"))</li>
        }
    </ul>
}

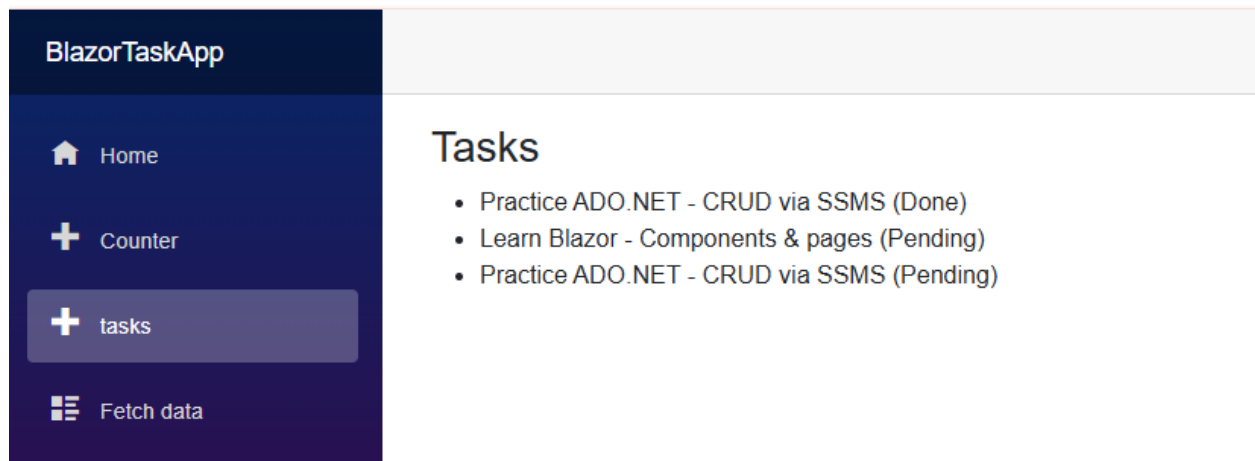
```

```

@code {
    private List<TaskModel>? tasks;
    private string? error;

    protected override async Task OnInitializedAsync()
    {
        try
        {
            tasks = await taskService.GetAllTasksAsync();
        }
        catch (Exception ex)
        {
            error = $"Error loading tasks: {ex.Message}";
        }
    }
}

```



Perform CRUD Operations:

Add task:

```

1 reference
public async Task AddTaskAsync(TaskModel task)
{
    using SqlConnection conn = new(_connectionString);
    await conn.OpenAsync();
    var cmd = new SqlCommand("INSERT INTO Tasks (Title, Description, IsCompleted) VALUES (@Title, @Description, @IsCompleted)", conn);
    cmd.Parameters.AddWithValue("@Title", task.Title);
    cmd.Parameters.AddWithValue("@Description", task.Description ?? "");
    cmd.Parameters.AddWithValue("@IsCompleted", task.IsCompleted);
    await cmd.ExecuteNonQueryAsync();
}

```

Update Task:

```

1 reference
public async Task UpdateTaskAsync(TaskModel task)
{
    using SqlConnection conn = new(_connectionString);
    await conn.OpenAsync();
    var cmd = new SqlCommand("UPDATE Tasks SET Title=@Title, Description=@Description, IsCompleted=@IsCompleted WHERE Id=@Id", conn);
    cmd.Parameters.AddWithValue("@Id", task.Id);
    cmd.Parameters.AddWithValue("@Title", task.Title);
    cmd.Parameters.AddWithValue("@Description", task.Description ?? "");
    cmd.Parameters.AddWithValue("@IsCompleted", task.IsCompleted);
    await cmd.ExecuteNonQuery();
}

```

Delete Task:

```

1 reference
public async Task DeleteTaskAsync(int id)
{
    using SqlConnection conn = new(_connectionString);
    await conn.OpenAsync();
    var cmd = new SqlCommand("DELETE FROM Tasks WHERE Id=@Id", conn);
    cmd.Parameters.AddWithValue("@Id", id);
    await cmd.ExecuteNonQuery();
}

```

Lab Task for Students

Build an Employee Management System using ADO.NET in Blazor Server

Objective: to connect to a database, retrieve employee data, and perform CRUD operations.

Task Instructions:

1. Create a new Blazor Server app named EmployeeManager.
2. Setup the SQL Server database with the given script.
3. Implement a service using ADO.NET to:
 - Fetch all employee records.
 - Add a new employee.
 - Update existing employee details.
 - Delete an employee.
4. Design a Razor page to manage employees using the service.
5. Bonus: Add validation and user feedback messages.

